CampusConnect

Documentation technique

Entraide et partage entre étudiants — backend Spring Boot, frontend Next.js, scénario de test complet

Projet	Projet au choix 01 — React & Next.js — sujet « CampusConnect »
Établissement	TEK-UP University — Cycle ingénieur, Génie logiciel (GLSI)
Équipe	Arbi Hazbri · Iheb Eddine Saafi
Version du document	1.0 — 2 novembre 2026
Dépôt	`github.com/arbihazbri/campusconnect`

> Ce document décrit l'application telle qu'elle est dans le dépôt local à la date indiquée : architecture, modèle de données, API, sécurité, interface, démarrage, puis un scénario pas à pas qui couvre toutes les fonctionnalités et les règles métier exigées par l'énoncé.

Sommaire

1. Présentation et périmètre

2. Architecture générale

3. Backend Spring Boot

3.1 Organisation du code · 3.2 Modèle de données · 3.3 Cycle de vie d'une demande · 3.4 API REST · 3.5 Sécurité · 3.6 Gestion des erreurs et concurrence · 3.7 Configuration · 3.8 Tests automatisés

4. Frontend Next.js

4.1 Organisation du code · 4.2 Authentification et relais d'API · 4.3 Protection des pages · 4.4 Pages · 4.5 Composants réutilisables · 4.6 Design, icônes, mode sombre · 4.7 Validation, états et accessibilité

5. Installation et démarrage (start.bat / stop.bat, IntelliJ)

6. Scénario de test complet

7. Limites connues et pistes d'amélioration

8. Déclaration d'utilisation d'outils d'IA

1 Présentation et périmètre

CampusConnect est une plateforme d'entraide : un étudiant publie une demande d'aide, d'autres étudiants proposent leur aide, l'auteur en accepte une, la séance a lieu puis les deux participants s'évaluent. Un administrateur modère les publications signalées.

Rôles

Rôle	Ce qu'il peut faire
Visiteur	Voir la page d'accueil, créer un compte, se connecter.
Étudiant (`STUDENT`)	Gérer son profil, publier et modifier ses demandes, rechercher, proposer son aide, accepter ou refuser les propositions reçues, clôturer une séance, évaluer, signaler une publication, consulter son tableau de bord.
Administrateur (`ADMIN`)	Consulter les publications signalées, masquer ou rétablir une publication, rejeter des signalements. Le compte est créé au démarrage à partir de `ADMIN_EMAIL` / `ADMIN_PASSWORD` .

Couverture des fonctionnalités obligatoires

Exigence de l'énoncé	Réalisation	Statut
Inscription, connexion, profil (spécialité, niveau, compétences)	`/api/auth/*` , `/api/users/me` · pages `/register` , `/login` , `/profile`	Fait
Publication d'une demande (titre, description, matière, disponibilité)	`POST /api/requests` · page `/requests/new` (+ niveau)	Fait
Recherche et filtrage par matière, niveau et statut	`GET /api/requests?subjectId&level&status&page` · filtres conservés dans l'URL	Fait
Proposition d'aide par un autre étudiant	`POST /api/requests/{id}/offers`	Fait
Acceptation ou refus par l'auteur	`POST /api/offers/{id}/accept|reject`	Fait
Suivi : ouverte → aide acceptée → terminée	`POST /api/requests/{id}/complete` · frise de suivi	Fait
Évaluation après une séance terminée	`POST|GET /api/requests/{id}/ratings`	Fait
Tableau de bord personnel	`GET /api/dashboard` · page `/dashboard`	Fait
Espace administrateur des publications signalées	`/api/requests/{id}/reports` , `/api/admin/*` · page `/admin`	Fait

Critères de validation

Critère	Où il est garanti
Un étudiant publie, un autre propose son aide	`OfferService.create` refuse l'auteur (403).

Critère	Où il est garanti
Seul l'auteur accepte une proposition et modifie sa demande	`OfferService.accept/reject`, `HelpRequestService.update` (403 sinon).
Une demande clôturée ne reçoit plus de propositions	Toute demande non `OUVERTE` refuse les propositions (409).
Évaluation réservée aux participants, après la clôture	`SessionService.rate` : 403 hors participants, 409 avant `TERMINEE`, 409 en double.
Filtres fonctionnels, données persistantes	Requête JPQL filtrée et paginée, stockage PostgreSQL, filtres dans l'URL.
Actions interdites bloquées côté serveur	Spring Security (rôles) + contrôles dans chaque service ; le frontend ne fait que masquer les boutons.

2 Architecture générale

L'application est découpée en trois couches. Le navigateur ne parle qu'au serveur Next.js ; celui-ci sert les pages et relaie les appels `/api/...` vers Spring Boot en ajoutant le jeton de connexion, ce qui évite tout problème de CORS et garde le jeton hors de portée du JavaScript.

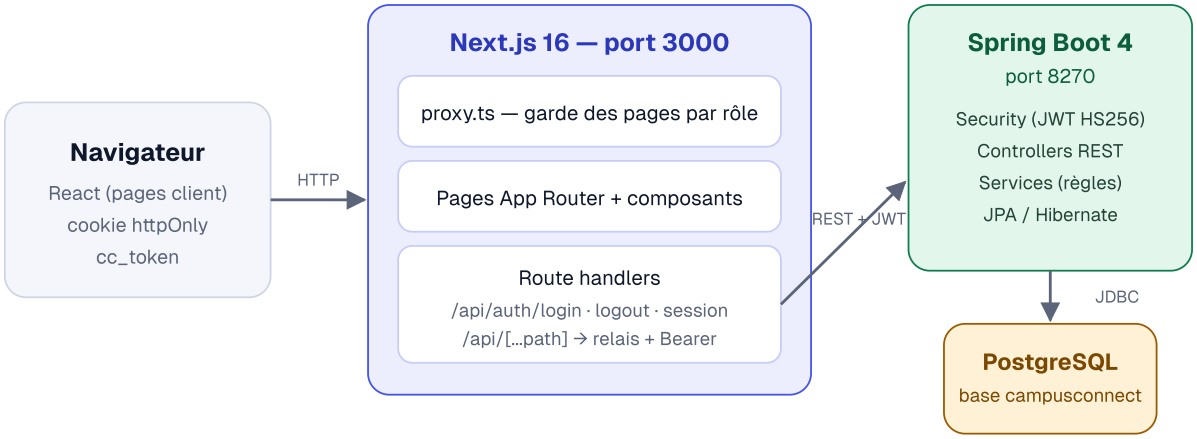


Figure 1 — Flux d'une requête : navigateur → Next.js (garde + relais) → Spring Boot → PostgreSQL

Pile technique

Couche	Technologie	Rôle
Frontend	Next.js 16.4 (App Router), React 19.3, TypeScript 5	Pages, routage, rendu, garde des pages (`proxy.ts`), relais d'API.
Icônes, police	Ionicons 8 (paquet npm), Geist (paquet npm)	Icônes SVG intégrées, police auto-hébergée (aucun appel réseau au build).

Couche	Technologie	Rôle
Backend	Java 21, Spring Boot 4.1.1 (Web MVC, Data JPA, Validation, Security, OAuth2 Resource Server)	API REST, règles métier, authentification JWT.
Données	PostgreSQL 16+ (testé avec 18), Hibernate 7	Persistance ; schéma créé et mis à jour automatiquement (`ddl-auto=update`).
Tests	JUnit 6, AssertJ, `@SpringBootTest` + client HTTP	92 tests d'intégration exécutés contre la vraie base.
Outillage	Maven (wrapper), npm, Git, IntelliJ IDEA	Build, scripts `start.bat` / `stop.bat` , configurations de lancement.

Arborescence du dépôt

```
campusconnect/
├── backend/                API Spring Boot (pom.xml, mvnw.cmd, src/main, src/test)
│   ├── .env                configuration locale (NON versionnée)
│   └── .env.example        modèle de configuration
├── frontend/               application Next.js (package.json, src/app, src/components,
src/lib)
├── docs/                   documentation (dont api-suivi-evaluation-admin.md)
├── .run/                   configurations IntelliJ « Start / Stop CampusConnect »
├── package.json            raccourcis npm : run start / run stop
├── start.bat               démarre backend + frontend
└── stop.bat                arrête backend + frontend
```

3 Backend Spring Boot

3.1 Organisation du code

Le code suit une architecture en couches classique, dans le paquet `com.campusconnect.backend` :

Paquet	Contenu
`entity`	Entités JPA : `User` , `Subject` , `HelpRequest` , `HelpOffer` , `Rating` , `Report` et les énumérations `Role` , `Level` , `RequestStatus` , `OfferStatus` , `ReportStatus` .
`repository`	Interfaces Spring Data JPA, requêtes JPQL (recherche filtrée, verrou `FOR UPDATE` , agrégats du tableau de bord).
`service`	Règles métier et transactions : `AuthService` , `JwtService` , `UserService` , `SubjectService` , `HelpRequestService` , `OfferService` , `SessionService` , `DashboardService` , `ReportService` .
`controller`	Contrôleurs REST minces : ils lisent l'identifiant de l'utilisateur dans le JWT et délèguent au service.
`dto`	Records d'entrée (validés par Bean Validation) et de sortie (aucune entité n'est exposée directement).

Paquet	Contenu
`security`	`SecurityConfig` (règles d'accès, réponses 401/403 en JSON) et `JwtConfig` (encodeur/décodeur HS256).
`exception`	Exceptions métier et `GlobalExceptionHandler` qui les traduit en codes HTTP.
`config`	`PasswordConfig` (BCrypt) et `DataInitializer` (10 matières + compte administrateur au démarrage).

3.2 Modèle de données

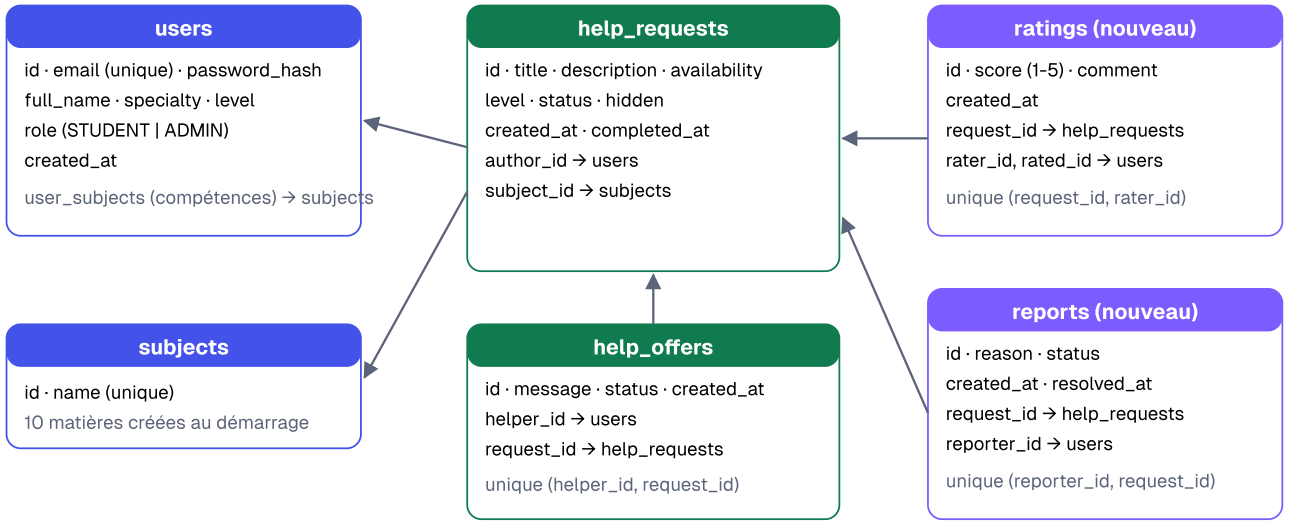


Figure 2 — Tables et relations (les flèches pointent vers la table référencée)

Énumération	Valeurs
`Role`	`STUDENT` , `ADMIN`
`Level`	`L1` , `L2` , `L3` , `M1` , `M2`
`RequestStatus`	`OUVERTE` → `AIDE_ACCEPTEE` → `TERMINEE`
`OfferStatus`	`EN_ATTENTE` , `ACCEPTEE` , `REFUSEE`
`ReportStatus`	`EN_ATTENTE` , `TRAITE` (publication masquée), `REJETE` (signalement non fondé)

> Une publication masquée par l'administrateur (`hidden = true`) n'est pas supprimée : elle disparaît de la liste, du détail et de toutes les actions des étudiants (réponse 404), mais reste visible par son auteur dans son tableau de bord avec la mention « Masquée par la modération ».

3.3 Cycle de vie d'une demande



Figure 3 — Transitions autorisées ; toute autre transition est refusée avec un code 409

Les **participants** d'une séance sont l'auteur de la demande et l'aidant dont la proposition a été acceptée. Chacun peut évaluer l'autre une seule fois (note de 1 à 5, commentaire facultatif de 500 caractères maximum), uniquement quand la demande est `TERMINEE`.

3.4 API REST

Toutes les routes sont préfixées par `/api`. Sauf mention « public », elles exigent l'en-tête `Authorization: Bearer <jeton>`. Les routes marquées NOUVEAU ont été ajoutées pour compléter le projet.

Méthode	Route	Accès	Description et réponses d'erreur
GET	`/health`	public	État du serveur.
POST	`/auth/register`	public	Inscription (email, mot de passe 8-64, nom). 201 ; 400 validation ; 409 email déjà utilisé (insensible à la casse).
POST	`/auth/login`	public	Connexion : renvoie `token`, durée et profil. 401 identique que l'email existe ou non (pas d'énumération des comptes).
GET PUT	`/users/me`	connecté	Lire / modifier son profil (nom, spécialité, niveau, compétences ≤ 20). 400 si une matière est inconnue.
GET	`/subjects`	connecté	Liste des matières.
POST	`/requests`	STUDENT	Publier une demande. 201 ; 400 validation (titre 5-100, description 10-2000, disponibilité ≤ 255).
GET	`/requests?subjectId&level&status&page&size`	connecté	Liste paginée (taille ≤ 50), plus récentes d'abord, sans les demandes masquées.
GET PUT	`/requests/{id}`	connecté	Détail ; modification par l'auteur seulement (403), demande ouverte seulement (409). 404 si inconnue ou masquée.
POST	`/requests/{id}/offers`	STUDENT	Proposer son aide (message 5-500). 403 sur sa propre demande ; 409 si non ouverte ou déjà proposé.
GET	`/requests/{id}/offers`	auteur	Propositions reçues, par ordre d'arrivée. 403 pour les autres.

Méthode	Route	Accès	Description et réponses d'erreur
POST	/offers/{id}/accept	auteur	Accepte : demande → `AIDE_ACCEPTEE`, autres propositions en attente → `REFUSEE`. 409 si plus ouverte ou déjà traitée.
POST	/offers/{id}/reject	auteur	Refuse une proposition ; la demande reste ouverte.
POST NOUVEAU	/requests/{id}/complete	auteur	Clôture : `AIDE_ACCEPTEE` → `TERMINEE`. 403 non-auteur ; 409 si encore ouverte ou déjà terminée.
POST NOUVEAU	/requests/{id}/ratings	participant	Évaluer l'autre participant (score 1-5, commentaire ≤ 500). 201 ; 403 non-participant ; 409 avant clôture ou en double.
GET NOUVEAU	/requests/{id}/ratings	participant	`{ canRate, ratings[] }` : indique au frontend s'il faut afficher le formulaire.
POST NOUVEAU	/requests/{id}/reports	STUDENT	Signaler une publication (motif 5-500). 403 sur sa propre demande ; 409 si déjà signalée par lui.
GET NOUVEAU	/dashboard	STUDENT	Statistiques, demandes publiées, aides proposées, séances réalisées, note moyenne.
GET NOUVEAU	/admin/reports?pendingOnly	ADMIN	Publications signalées regroupées par demande (par défaut : seulement celles à traiter).
POST NOUVEAU	/admin/requests/{id}/hide	ADMIN	Masque la publication ; signalements en attente → `TRAITE`. 409 si déjà masquée.
POST NOUVEAU	/admin/requests/{id}/restore	ADMIN	Rend la publication visible. 409 si elle n'est pas masquée.
POST NOUVEAU	/admin/requests/{id}/dismiss	ADMIN	Rejette les signalements en attente → `REJETE`. 409 s'il n'y en a aucun.

Format commun des erreurs :

```
{ "status": 409, "error": "Conflict", "message": "Cette demande est déjà terminée",
"fieldErrors": {} }
{ "status": 400, "error": "Bad Request", "message": "Les données envoyées sont invalides",
  "fieldErrors": { "score": "La note doit être comprise entre 1 et 5" } }
```

3.5 Sécurité

- **Mots de passe** hachés avec BCrypt. À la connexion, un hachage factice est vérifié quand l'email n'existe pas, pour que le temps de réponse ne révèle pas l'existence d'un compte.
- **Jeton JWT** signé en HS256 avec `JWT_SECRET` (32 caractères minimum, sans valeur par défaut). Il contient l'identifiant (`sub`), l'email, le rôle et l'expiration (120 minutes par défaut).
- **Spring Security** en mode sans session (`STATELESS`), CSRF désactivé (API à jeton). Règles d'accès dans `SecurityConfig` :

```
/api/auth/**, /api/health                       → public
/api/admin/**                                   → rôle ADMIN
POST /api/requests, /api/requests/*/offers,
     /api/requests/*/reports                    → rôle STUDENT
/api/dashboard                                  → rôle STUDENT
toute autre route                               → utilisateur authentifié
```

- **Contrôles métier dans les services** : être l'auteur, être participant, statut de la demande, doublons. Ces règles s'appliquent même si quelqu'un appelle l'API sans passer par l'interface.
- **Réponses 401 / 403 en JSON** (« Authentification requise », « Accès refusé ») produites par le point d'entrée et le gestionnaire d'accès refusé.
- **Aucun secret dans le dépôt** : mots de passe et clé lus dans des variables d'environnement ; `backend/.env` est ignoré par Git.

3.6 Gestion des erreurs et concurrence

Exception	Code	Utilisation
Validation Bean (`@Valid`), `BusinessValidationException`	400	Champs invalides, avec le détail par champ dans `fieldErrors` .
`InvalidCredentialsException`	401	Email ou mot de passe incorrect.
`ForbiddenOperationException`	403	Action réservée à l'auteur, à un participant, ou interdite sur sa propre demande.
`ResourceNotFoundException`	404	Ressource inconnue ou demande masquée.
`ConflictException` , `EmailAlreadyUsedException`	409	État incompatible (déjà traité, plus ouverte, doublon).

Concurrence : les opérations qui changent l'état d'une demande (proposer, accepter, refuser, modifier, clôturer, masquer) chargent la demande avec un verrou pessimiste (`SELECT … FOR UPDATE`). Deux acceptations simultanées ne peuvent donc pas réussir toutes les deux : un test le vérifie (une réponse 200, l'autre 409). Les contraintes d'unicité en base (proposition, évaluation, signalement) servent de dernier rempart : une violation est traduite en 409.

3.7 Configuration

Variable	Obligatoire	Rôle (valeur par défaut)
`DB_PASSWORD`	oui	Mot de passe de `campusconnect_user` .
`JWT_SECRET`	oui	Clé de signature, ≥ 32 caractères.
`ADMIN_EMAIL` / `ADMIN_PASSWORD`	recommandé	Compte administrateur créé au démarrage s'il n'existe pas.
`DB_HOST` / `DB_PORT` / `DB_NAME` / `DB_USERNAME`	non	`localhost` / `5432` / `campusconnect` / `campusconnect_user` .
`SERVER_PORT`	non	Port HTTP du backend (`8270`). `start.bat` le règle à partir de `BACKEND_PORT` .
`JWT_EXPIRATION_MINUTES`	non	Durée de validité du jeton (`120`).

Variable	Obligatoire	Rôle (valeur par défaut)
`BACKEND_URL`	non (frontend)	Adresse du backend vue par Next.js (`http://localhost:8270`).
`JAVA_HOME` / `MAVEN_CMD`	non (scripts)	JDK 21+ et Maven utilisés par `start.bat` (sinon `mvnw.cmd`).

> Le port du backend est passé de 8080 à **8270** : sur le poste de développement, le port 8080 était déjà occupé par un autre serveur, qui répondait 401 à la place de l'API.

3.8 Tests automatisés

92 tests d'intégration (`@SpringBootTest` sur un port aléatoire) appellent l'API en HTTP contre la vraie base PostgreSQL ; chaque test supprime les données qu'il a créées. Résultat obtenu : **Tests run: 92, Failures: 0, Errors: 0 — BUILD SUCCESS**.

Classe	Tests	Ce qui est vérifié
AuthRegistrationTests / AuthLoginTests	4 / 5	Inscription, doublon d'email, JSON invalide, connexion, 401 identique, accès admin refusé.
ProfileTests	6	Lecture et mise à jour du profil, matières inconnues, isolement entre comptes.
HelpRequestTests / ListTests / UpdateTests	8 / 9 / 9	Publication, détail, filtres, pagination, demandes masquées, droits de modification.
OfferTests / OfferActionTests	10 / 11	Propositions, doublons, acceptation, refus, acceptation concurrente.
SessionCompletionTests NOUVEAU	6	Clôture par l'auteur seulement, demande ouverte refusée, double clôture, plus de propositions ni de modification.
RatingTests NOUVEAU	7	Évaluation mutuelle, avant clôture refusée, non-participants et aidant refusé exclus, une seule fois, validation.
DashboardTests NOUVEAU	4	Tableau vide, contenu des deux participants, demande masquée signalée, accès réservé aux étudiants.
ReportModerationTests NOUVEAU	8	Règles de signalement, regroupement admin, masquer / rétablir / rejeter, 404, accès interdit aux étudiants.
Autres (contexte, initialisation, dépôt)	5	Démarrage, matières par défaut, BCrypt, unicité de l'email en base.

```
cd backend
mvnw.cmd test            (ou le mvn.cmd indiqué dans MAVEN_CMD ; les variables DB_* et
JWT_SECRET doivent être définies)
```

4 Frontend Next.js

4.1 Organisation du code

```
frontend/src/
├── proxy.ts                    garde des pages selon la connexion et le rôle
├── app/                        App Router : un dossier = une route
│   ├── layout.tsx              en-tête, pied de page, fournisseur de session, thème
│   ├── page.tsx                accueil
│   ├── login/ · register/      authentification (AuthShell + formulaire)
│   ├── profile/                profil et compétences
│   ├── requests/               liste filtrée · new/ · [id]/ (détail) · [id]/edit/
│   ├── dashboard/              tableau de bord étudiant
│   ├── admin/                  publications signalées
│   ├── api/auth/login|logout|session/route.ts    connexion par cookie httpOnly
│   ├── api/[...path]/route.ts  relais vers Spring Boot avec le jeton
│   ├── icon.svg · favicon.ico · apple-icon.png   logo
│   └── globals.css             système de design (clair / sombre)
├── components/                 composants réutilisables (Icon, Avatar, Field, States, …)
└── lib/                        types de l'API, client HTTP, formats, icônes des matières
```

4.2 Authentification et relais d'API

1. **Connexion** : le formulaire appelle `POST /api/auth/login` (route Next). Celle-ci appelle Spring, puis place le jeton dans un cookie `cc_token` **httpOnly** (illisible par JavaScript, donc protégé contre le vol par script), `SameSite=Lax`, `Secure` en production, de même durée que le jeton. Le navigateur ne reçoit que le profil.
2. **Appels d'API** : les pages appellent `/api/...`. La route `api/[...path]` lit le cookie et transmet la requête à Spring avec `Authorization: Bearer`. Les routes publiques (`auth`, `health`) partent sans jeton, pour qu'un vieux cookie ne bloque jamais l'inscription. Si Spring répond 401, le cookie est effacé.
3. **Session** : `GET /api/auth/session` renvoie `{ user }` ou `{ user: null }` sans erreur ; le composant `AuthProvider` l'utilise au chargement pour l'en-tête et les pages.
4. **Déconnexion** : `POST /api/auth/logout` supprime le cookie.
5. **Session expirée** : le client HTTP (`lib/api.ts`) redirige vers `/login?next=…&expired=1` et l'utilisateur revient sur sa page après reconnexion.

4.3 Protection des pages

`proxy.ts` (le « middleware » de Next.js 16) s'exécute avant chaque page. Il lit le rôle et l'expiration contenus dans le jeton — sans vérifier la signature, ce que fait Spring à chaque appel — et applique les règles suivantes :

Situation	Effet
Visiteur sur une page protégée	Redirection vers `/login?next=<page>`
Connecté sur `/login` ou `/register`	Redirection vers `/dashboard` (étudiant) ou `/admin` (admin)
Étudiant sur `/admin`	Redirection vers `/dashboard`
Admin sur `/dashboard`, `/requests/new`, `/requests/{id}/edit`	Redirection vers `/admin`

Situation	Effet
Jeton expiré ou illisible	Cookie supprimé, traité comme un visiteur

> Le frontend masque les boutons qui ne concernent pas l'utilisateur, mais ce n'est qu'un confort : toutes les règles sont vérifiées par le backend.

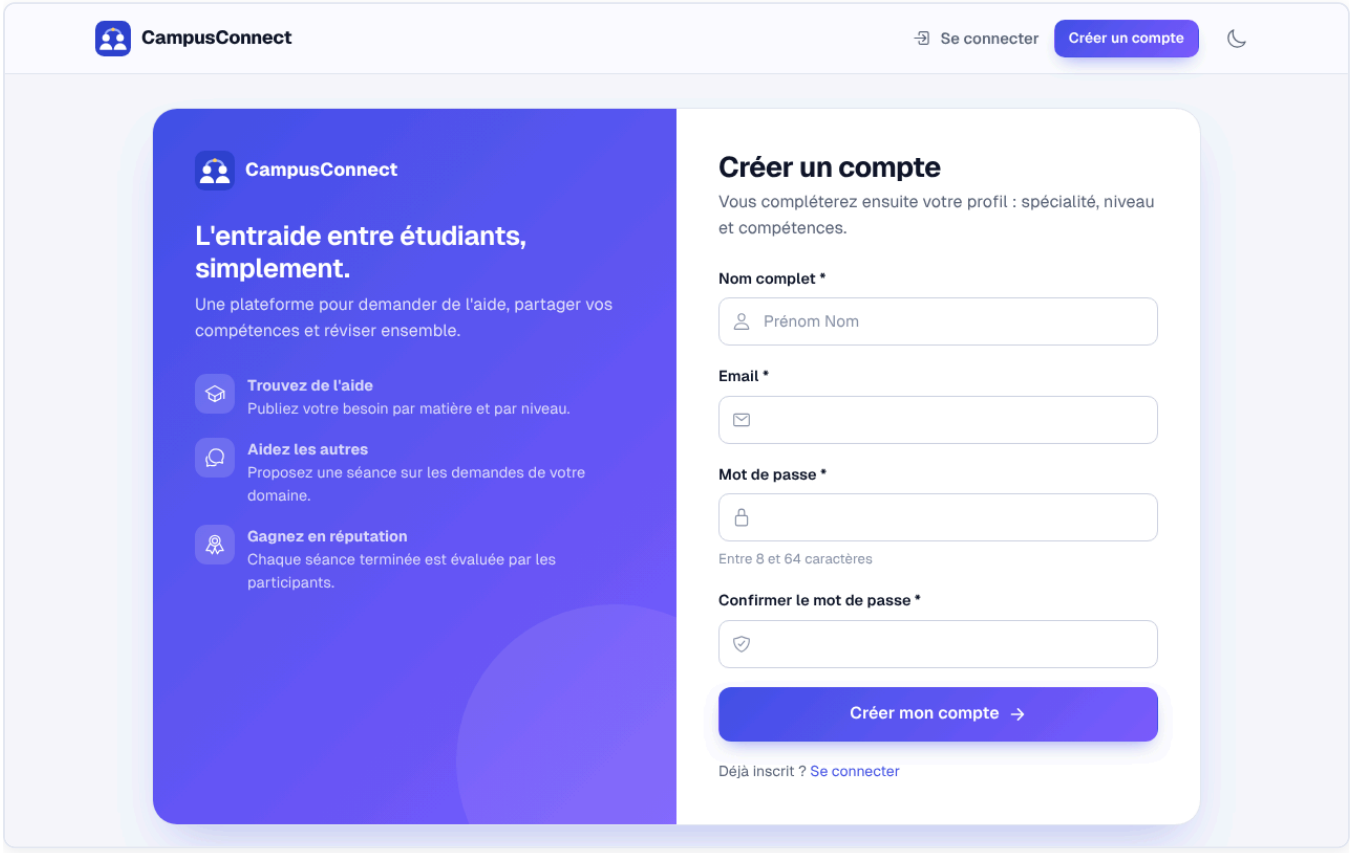
4.4 Pages

Route	Accès	Contenu
`/`	public	Présentation, fonctionnalités, « comment ça marche », état du serveur.
`/register`	visiteur	Inscription ; connexion automatique puis redirection vers le profil.
`/login`	visiteur	Connexion ; retour à la page demandée (`next`), message si session expirée.
`/profile`	connecté	Nom, spécialité, niveau, compétences (cases à cocher avec icône de matière) et aperçu du profil.
`/requests`	connecté	Liste paginée, filtres matière / niveau / statut synchronisés avec l'URL (conservés au rechargement).
`/requests/new`	étudiant	Formulaire de publication validé côté client.
`/requests/{id}`	connecté	Détail, frise de suivi ; auteur : propositions, accepter / refuser, modifier, clôturer ; autre étudiant : proposer son aide, signaler ; participants : évaluations.
`/requests/{id}/edit`	étudiant	Modification par l'auteur d'une demande ouverte.
`/dashboard`	étudiant	Bandeau d'accueil, 4 statistiques, demandes publiées, aides proposées, séances réalisées avec bouton « Évaluer ».
`/admin`	admin	Publications signalées (à traiter / historique), masquer, rétablir, rejeter.

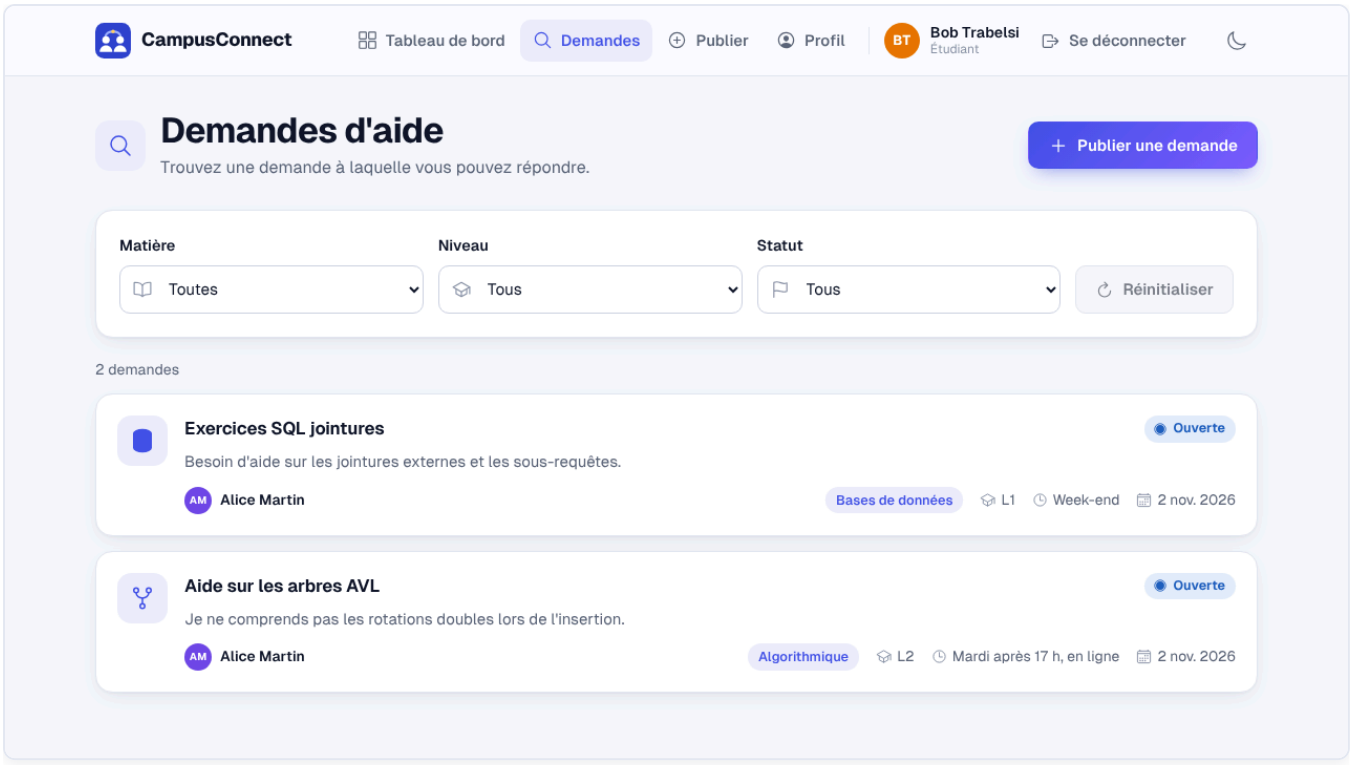
Aperçu de l'interface



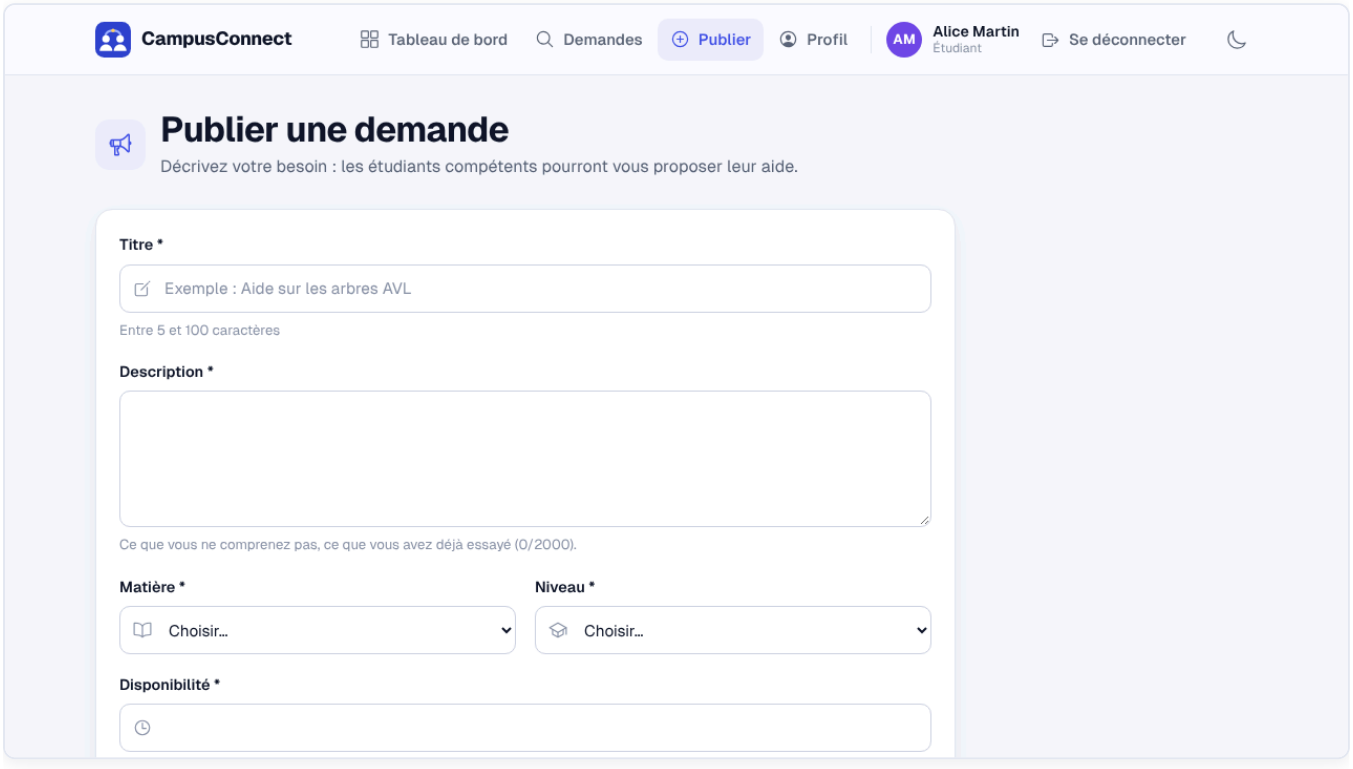
Accueil — présentation, fonctionnalités et appel à l'action



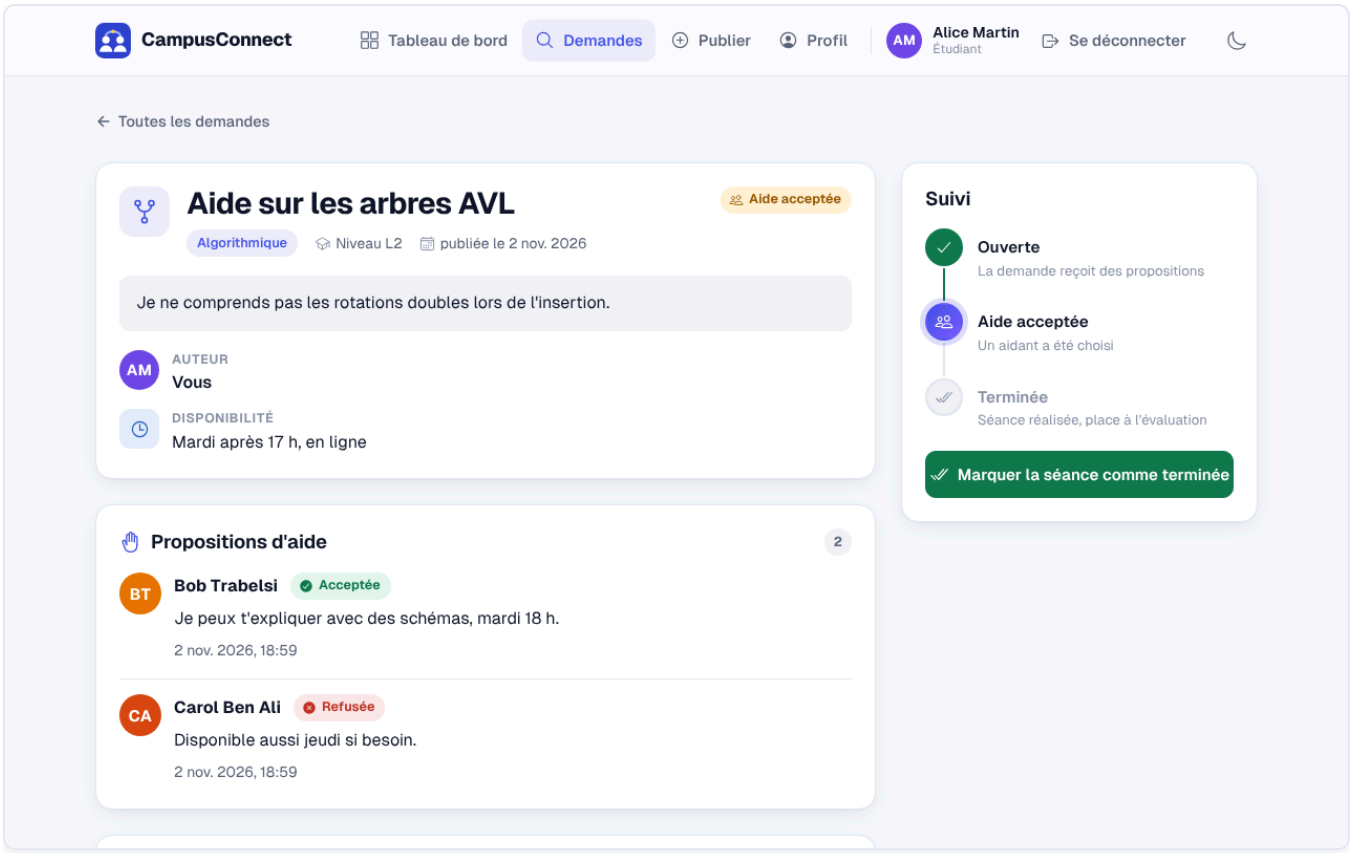
Inscription — présentation à gauche, formulaire validé à droite



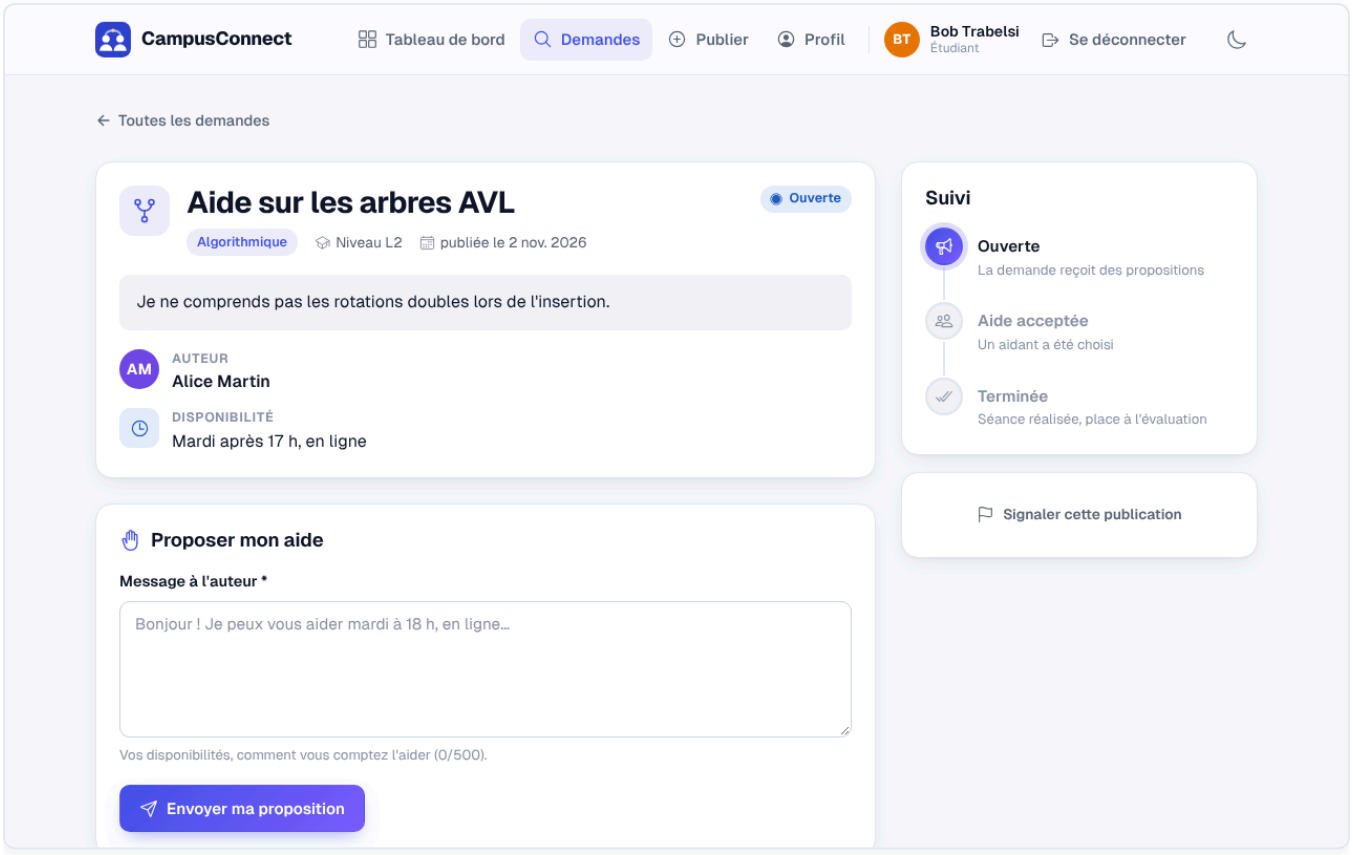
Liste des demandes — filtres matière / niveau / statut et cartes avec icône de matière



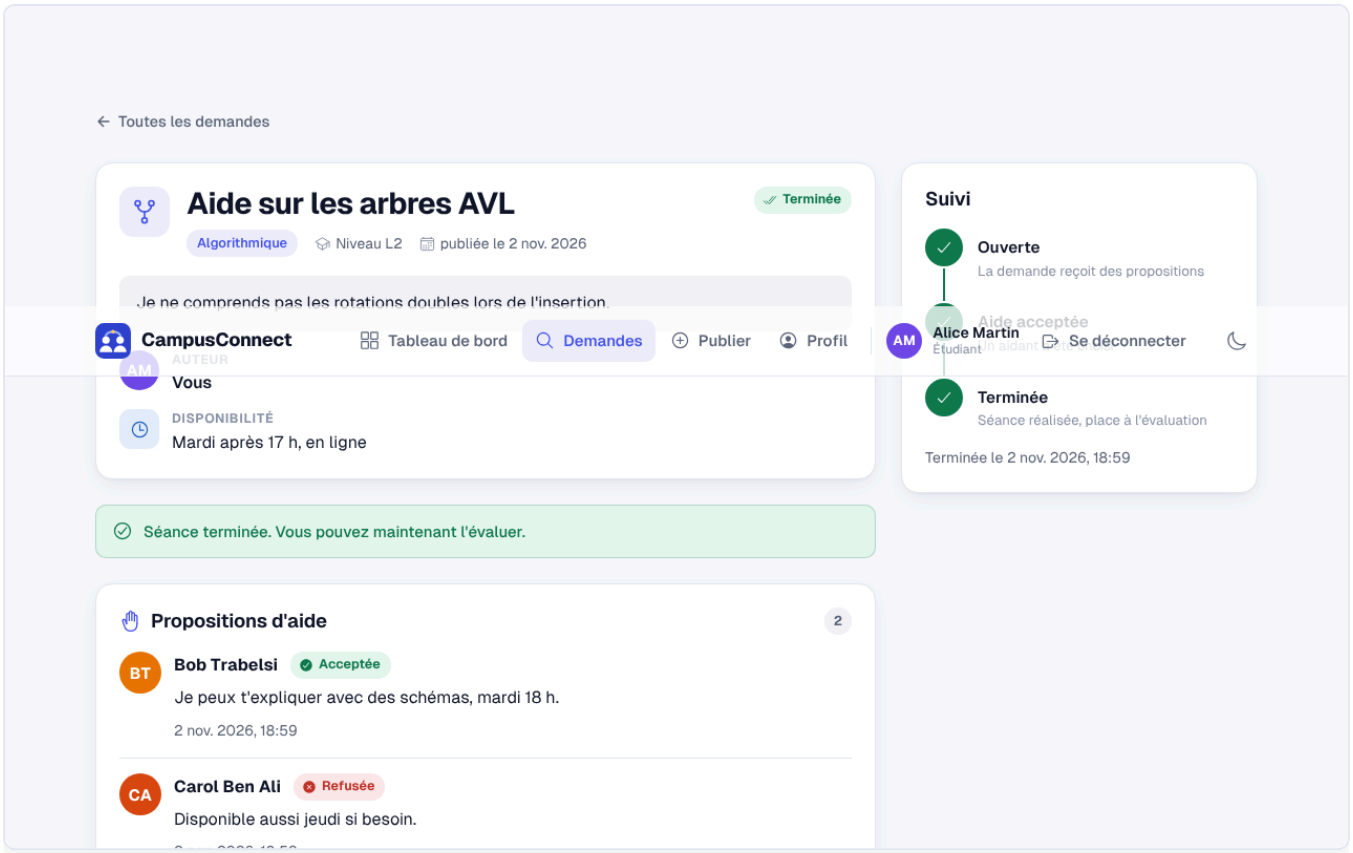
Publication d'une demande



Détail côté auteur — propositions acceptée / refusée et frise de suivi



Détail côté autre étudiant — proposer son aide et signaler



Séance terminée — évaluations des deux participants

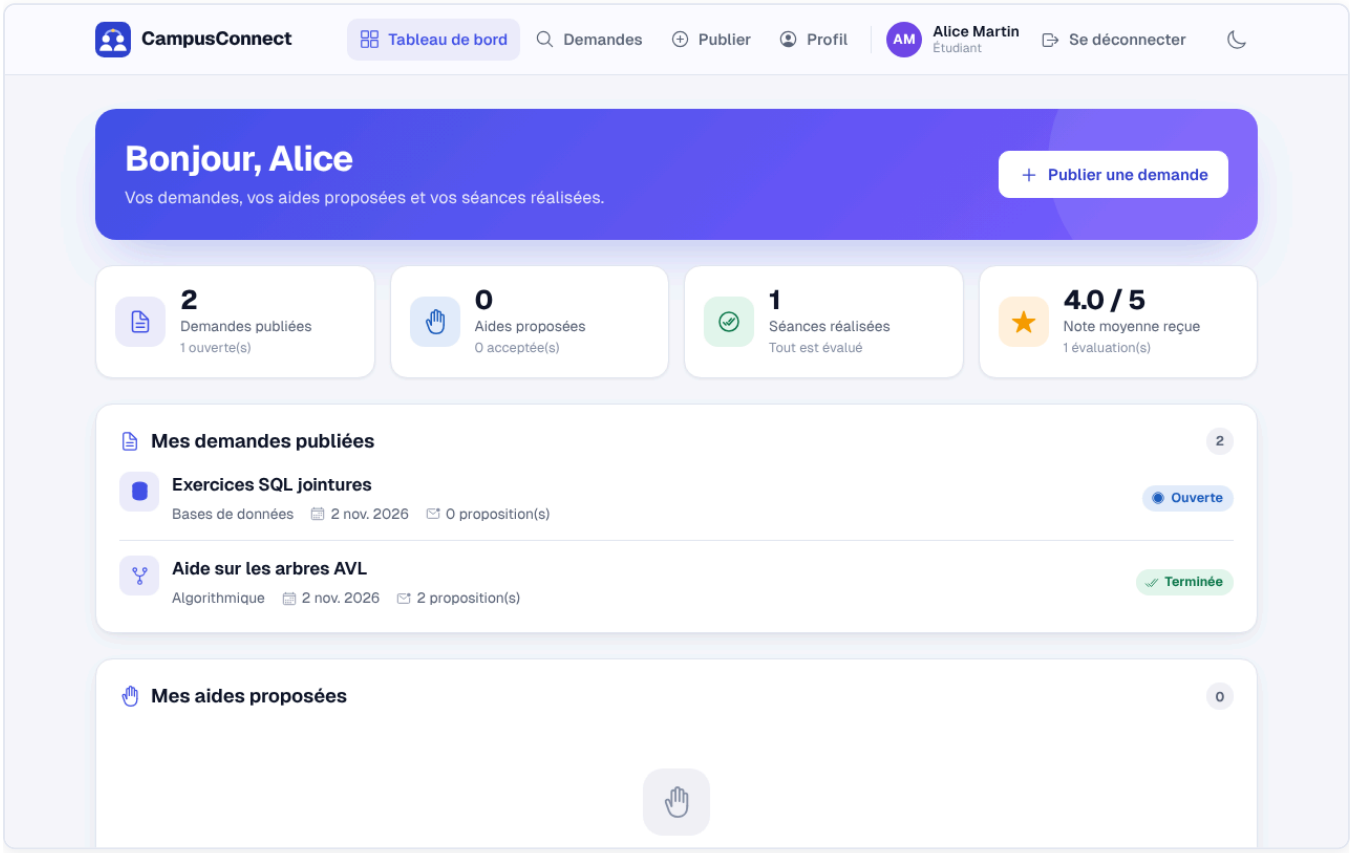
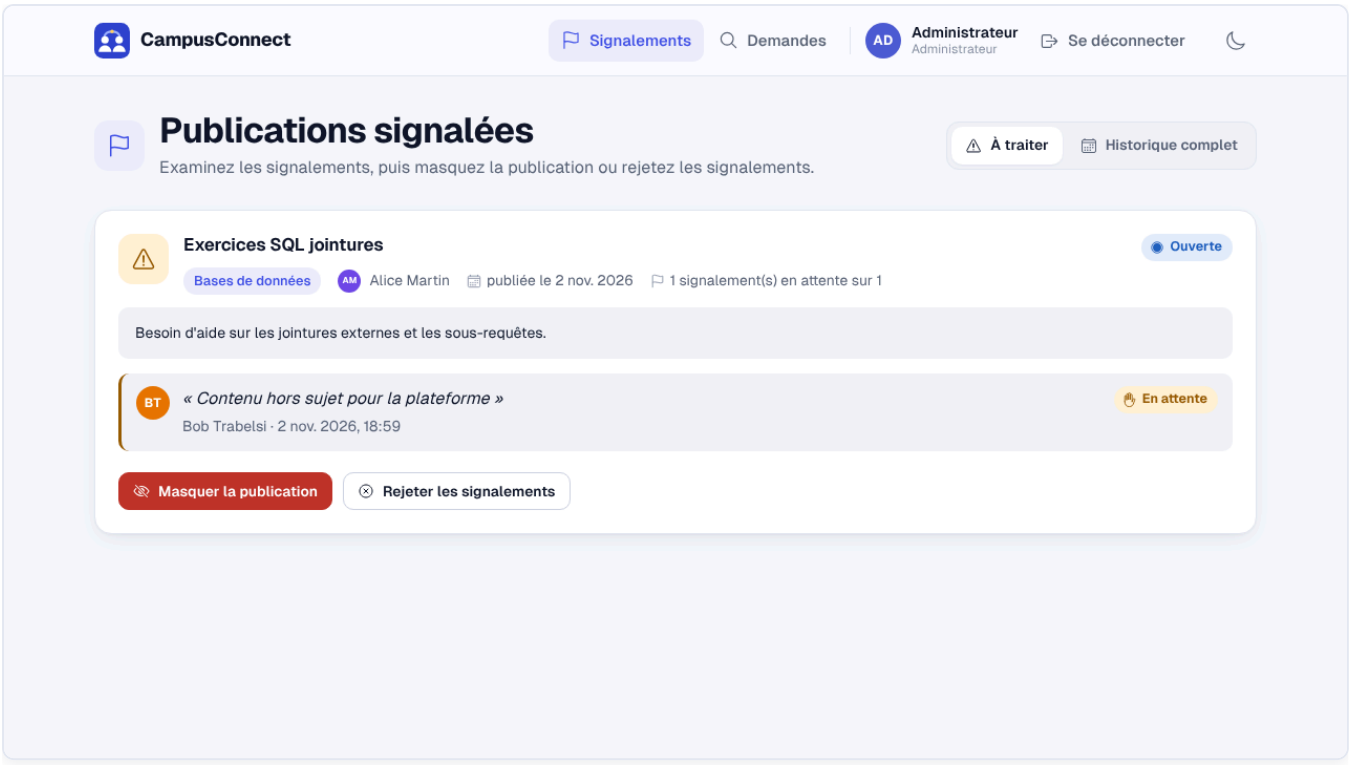
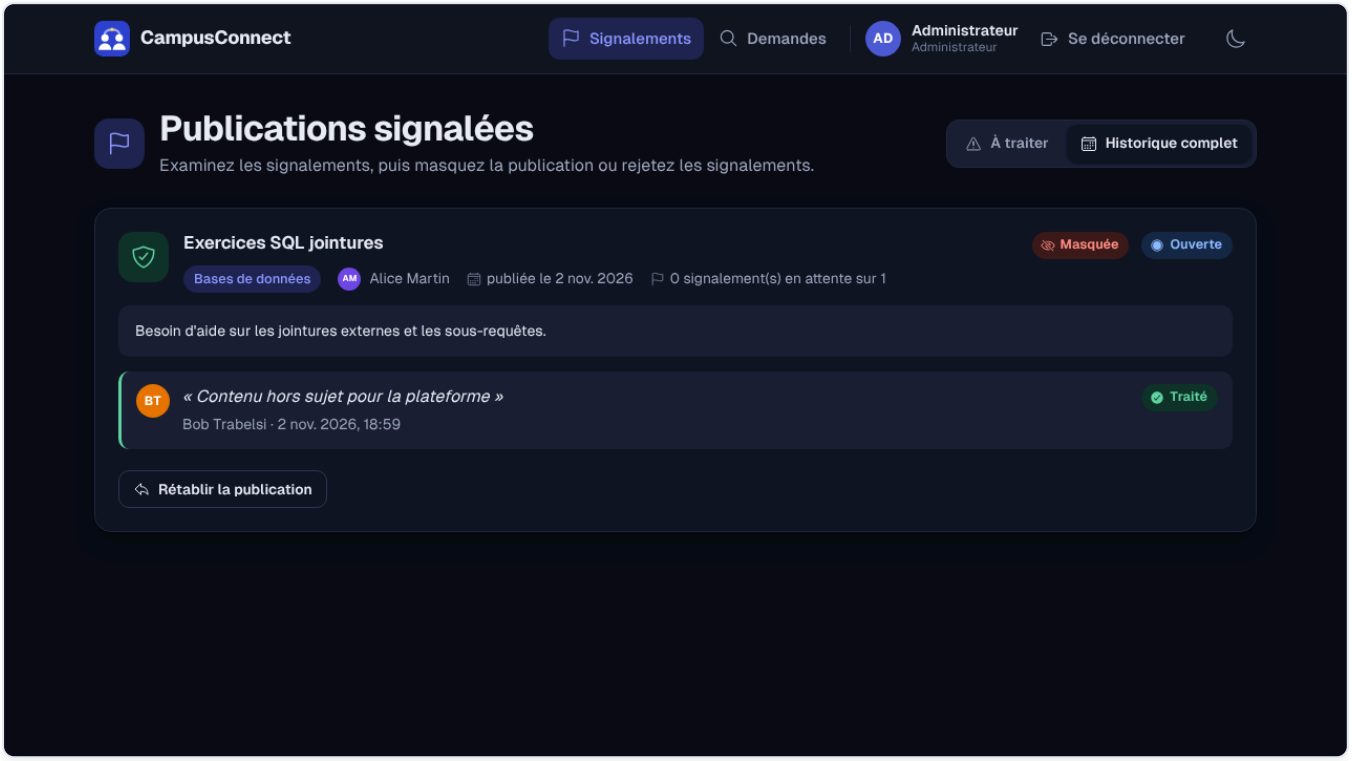


Tableau de bord — statistiques, demandes, aides et séances



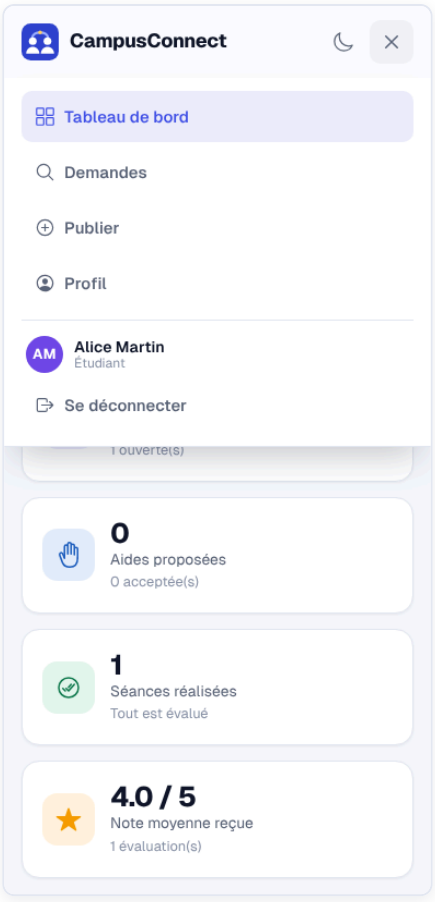
Espace administrateur — signalements à traiter



Historique de modération en mode sombre



Mobile — tableau de bord



Mobile — menu



Mobile — détail d'une demande

4.5 Composants réutilisables

Composant	Rôle
`AuthProvider`	Contexte de session : utilisateur courant, rechargement, déconnexion.
`SiteHeader`	En-tête : logo, navigation selon le rôle avec icônes, avatar, thème, menu mobile.
`AuthShell`	Mise en page des écrans de connexion et d'inscription (présentation + formulaire).
`PageHead`	Titre de page avec icône, sous-titre et actions.
`Field`	Champ accessible : libellé relié, icône, aide et erreur annoncées (`aria-describedby` , `aria-invalid`).
`Icon`	Icône Ionicons rendue en SVG intégré (couleur héritée, rendu serveur, aucun appel réseau).
`Avatar`	Pastille aux initiales, couleur stable dérivée du nom.
`Badges`	Badges de statut (demande, proposition, signalement) avec icône, et étoiles de notation.
`StatusTimeline`	Frise de suivi ouverte → aide acceptée → terminée.
`States`	États de chargement (squelettes, indicateur), d'erreur (avec « Réessayer »), de liste vide et messages.

Composant	Rôle
`useApiData`	Hook de chargement : données, erreur, code HTTP, rechargement, annulation à la navigation.
`RequestForm`	Formulaire partagé par la publication et la modification d'une demande.
`ThemeToggle · Logo · BackendStatus`	Bascule clair / sombre, logo SVG, indicateur d'état du serveur.

4.6 Design, icônes et mode sombre

- **Système de design** dans `globals.css` : jetons de couleurs, rayons et ombres déclarés une fois en clair et en sombre ; dégradé indigo → violet pour les actions principales ; cartes, badges, avatars et icônes en tuiles.
- **Icônes Ionicons** (paquet npm `ionicons`) : chaque matière a son icône (calculatrice, réseau, base de données…), ainsi que la navigation, les statuts et les actions.
- **Logo** original : deux étudiants reliés par un arc avec une étincelle — décliné en favicon, icône SVG et icône Apple.
- **Mode sombre** : suit le système au premier passage ; le bouton lune / soleil enregistre le choix (`localStorage`) et un petit script appliqué avant l'affichage évite le flash du mauvais thème.
- **Responsive** : grilles fluides, menu repliable sous 860 px ; vérifié sans défilement horizontal à 390 px.

4.7 Validation, états et accessibilité

- Chaque formulaire applique **les mêmes règles que le serveur** avant l'envoi (longueurs, champs obligatoires, email, confirmation du mot de passe, note de 1 à 5), puis affiche les erreurs renvoyées par l'API sous les champs concernés.
- Chaque chargement a ses **états** : squelettes ou indicateur, message d'erreur avec bouton « Réessayer », état vide avec action (ex. « Publier une demande »).
- **Accessibilité** : langue `fr`, lien « Aller au contenu », libellés reliés aux champs, focus visible, `aria-current` dans la navigation, messages annoncés (`role="alert"` / `status`), étoiles utilisables au clavier, animations désactivées si l'utilisateur le demande (`prefers-reduced-motion`).
- Les actions irréversibles (accepter, refuser, clôturer, masquer, rejeter) demandent une **confirmation**.

5 Installation et démarrage

Prérequis

Outil	Version	Vérification
JDK	21 ou plus	`java -version`
Node.js / npm	20+ / 10+	`node -v` , `npm -v`
PostgreSQL	16 ou plus	`Get-Service *postgres*` → *Running*
Git	2.x	`git --version`

1. Base de données (une seule fois)

```
& "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -h localhost -p 5432 -c "CREATE
ROLE campusconnect_user WITH LOGIN PASSWORD 'MotDePasse';"
& "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -h localhost -p 5432 -c "CREATE
DATABASE campusconnect OWNER campusconnect_user ENCODING 'UTF8';"
```

Les tables, les 10 matières et le compte administrateur sont créés au premier démarrage du backend.

2. Configuration : `backend\.env`

Copier `backend\.env.example` en `backend\.env` et le compléter (ce fichier n'est jamais commité) :

```
DB_PORT=5432
DB_PASSWORD=MotDePasse
ADMIN_EMAIL=admin@campusconnect.local
ADMIN_PASSWORD=Admin12345
JWT_SECRET=<au moins 32 caractères aléatoires>
JAVA_HOME=C:\chemin\vers\jdk-21          (facultatif)
MAVEN_CMD=C:\chemin\vers\maven\bin\mvn.cmd (facultatif, sinon mvnw.cmd)
BACKEND_PORT=8270
FRONTEND_PORT=3000
```

3. Démarrer et arrêter

start.bat

1. charge `backend\.env` et vérifie les variables obligatoires, npm, PostgreSQL et que le port est libre ;
2. installe les dépendances du frontend au premier lancement ;
3. ouvre deux fenêtres : « CampusConnect - backend » et « CampusConnect - frontend » ;
4. attend que l'API réponde, puis ouvre `http://localhost:3000`.

stop.bat

1. ferme les deux fenêtres ouvertes par `start.bat` avec tous leurs processus (Maven, Java, npm, Node) ;
2. par sécurité, libère les ports 8270 et 3000 s'ils sont encore occupés.

4. Depuis IntelliJ IDEA

Ouvrir le dossier racine `campusconnect`. Les configurations **Start CampusConnect** et **Stop CampusConnect** (type npm, fichiers dans `.run/`) apparaissent dans la liste en haut à droite ; elles exécutent `npm run start` / `npm run stop` du `package.json` racine, qui lancent les deux scripts. À défaut : *Edit Configurations… → + → npm*, `package.json` racine, commande `run`, script `start` (ou `stop`).

Comptes de démonstration

Compte	Identifiants
Administrateur	valeurs de `ADMIN_EMAIL` / `ADMIN_PASSWORD` (ex. `admin@campusconnect.local` / `Admin12345`)
Étudiants	à créer depuis `/register` (le scénario du chapitre 6 utilise Alice, Bob et Carol)

Dépannage

Symptôme	Solution
« Authentification requise » à l'inscription	Un autre serveur occupe le port du backend : vérifier `BACKEND_PORT` et que la fenêtre backend affiche *Tomcat started on port 8270*.
« Le serveur CampusConnect est injoignable »	Le backend n'est pas démarré ou a échoué : lire la fenêtre « CampusConnect - backend ».
Backend : *Connection to localhost:5434 refused*	PostgreSQL arrêté ou autre port : régler `DB_PORT`.
Backend : *password authentication failed*	`DB_PASSWORD` incorrect.
IntelliJ : *Interpreter not found*	Utiliser les configurations de type npm (et non Shell Script).

6 Scénario de test complet

Ce scénario parcourt toutes les fonctionnalités et vérifie chaque règle métier, cas d'erreur compris. Il demande **trois navigateurs ou fenêtres privées** pour être connecté en même temps avec trois comptes (ex. Chrome, une fenêtre privée Chrome, Edge). Cocher chaque ligne au fur et à mesure.

#	Qui	Action	Résultat attendu	
A. Accès et authentification				
1	Visiteur	Ouvrir `http://localhost:3000`.	Accueil avec logo ; en bas « Serveur CampusConnect : ok » (point vert).	☐
2	Visiteur	Aller directement sur `/dashboard`.	Redirection vers `/login?next=%2Fdashboard`.	☐
3	Visiteur	Cliquer « Se connecter » sans rien saisir.	« Saisissez un email valide » et « Le mot de passe est obligatoire ».	☐
4	Visiteur	Se connecter avec un email inconnu.	« Email ou mot de passe incorrect ».	☐
5	Alice	S'inscrire : Alice Martin, alice@test.tn, mot de passe `MotDePasse123` (essayer d'abord une confirmation différente).	Erreur « Les deux mots de passe ne correspondent pas », puis succès : connexion automatique, page Profil avec « Bienvenue ! ».	☐
6	Alice	Profil : spécialité « Génie logiciel », niveau L3, compétence Algorithmique → Enregistrer.	« Profil enregistré. » ; l'aperçu à droite se met à jour ; valeurs conservées après rechargement.	☐
7	Bob, Carol	Dans deux autres navigateurs, inscrire Bob Trabelsi (bob@test.tn) et Carol Ben Ali (carol@test.tn).	Comptes créés. Réessayer bob@test.tn en majuscules → « Cet email est déjà utilisé ».	☐
B. Publication, recherche et filtres				
8	Alice	« Publier » puis cliquer « Publier » sans rien remplir.	Erreurs sous chaque champ (titre 5-100, description 10-2000, matière, niveau, disponibilité).	☐
9	Alice	Publier « Aide sur les arbres AVL », Algorithmique, L2, « Mardi après 17 h, en ligne ».	Page détail, message « Votre demande est publiée », statut **Ouverte**, frise à l'étape 1, « Aucune proposition pour l'instant ».	☐
10	Alice	Publier « Exercices SQL jointures », Bases de données, L1.	Seconde demande créée.	☐
11	Alice	Sur la demande AVL, « Modifier la demande » : changer la disponibilité.	« Modifications enregistrées. »	☐
12	Bob	« Demandes » : filtre Matière = Algorithmique, puis recharger la page (F5).	Seule la demande AVL ; le filtre reste dans l'URL et est conservé au rechargement.	☐
13	Bob	Ajouter Statut = Terminée, puis « Réinitialiser ».	« Aucune demande ne correspond à ces filtres », puis les deux demandes réapparaissent.	☐
C. Propositions, acceptation, refus				

#	Qui	Action	Résultat attendu	
14	Bob	Ouvrir la demande AVL.	Pas de bouton « Modifier » ; formulaire « Proposer mon aide » et lien « Signaler ».	☐
15	Bob	Proposer : « Je peux t'expliquer avec des schémas, mardi 18 h. »	« Proposition envoyée ».	☐
16	Carol	Proposer son aide sur la même demande, puis recharger et proposer à nouveau.	1re fois : envoyée ; 2e fois : « Vous avez déjà proposé votre aide sur cette demande ».	☐
17	Alice	Essayer de proposer son aide sur sa propre demande.	Impossible : pas de formulaire (et l'API répondrait 403).	☐
18	Alice	Ouvrir la demande AVL : 2 propositions « En attente ». Cliquer « Accepter » sur Bob et confirmer.	Bob **Acceptée**, Carol **Refusée** automatiquement ; statut **Aide acceptée** ; frise à l'étape 2 ; bouton « Modifier » disparu.	☐
19	Carol	Recharger la demande AVL.	Plus de formulaire de proposition (demande non ouverte) ; pas de section évaluation (aidant refusé).	☐
20	Bob	Ouvrir la demande AVL.	Section évaluation : « L'évaluation sera possible quand l'auteur aura marqué la séance comme terminée ».	☐
D. Clôture et évaluation				
21	Alice	« Marquer la séance comme terminée » et confirmer.	« Séance terminée. Vous pouvez maintenant l'évaluer. » ; statut **Terminée** ; frise complète avec date de fin ; formulaire d'évaluation affiché.	☐
22	Alice	Cliquer « Envoyer mon évaluation » sans note.	« Choisissez une note de 1 à 5 ».	☐
23	Alice	Donner 5 étoiles, commentaire « Explications très claires, merci ! », envoyer.	L'évaluation s'affiche (Alice → Bob) ; le formulaire disparaît (une seule évaluation).	☐
24	Bob	Tableau de bord → « Séances réalisées » → « Évaluer » ; donner 4 étoiles.	Arrivée sur la section évaluation ; évaluation enregistrée (Bob → Alice).	☐
25	Carol	Rouvrir la demande AVL.	Aucune section évaluation : seuls les participants y ont accès.	☐
E. Tableau de bord				
26	Alice	Ouvrir le tableau de bord.	2 demandes publiées (1 ouverte), 1 séance réalisée « Aidé par Bob Trabelsi » marquée « Évaluée », note moyenne **4.0 / 5** (1 évaluation).	☐
27	Bob	Ouvrir le tableau de bord.	1 aide proposée (1 acceptée), statut de la demande « Terminée », 1 séance « Vous avez aidé Alice Martin », note moyenne 5.0 / 5.	☐
28	Carol	Ouvrir le tableau de bord.	1 aide proposée avec le badge **Refusée** ; aucune séance.	☐

#	Qui	Action	Résultat attendu	
F. Signalement et administration				
29	Bob	Ouvrir « Exercices SQL jointures » → « Signaler cette publication » ; motif « abc ».	« Le motif doit contenir entre 5 et 500 caractères ».	☐
30	Bob	Motif « Contenu hors sujet pour la plateforme » → envoyer.	« Merci, la publication a été signalée à l'administrateur. »	☐
31	Carol	Signaler la même demande (« Doublon d'une autre demande »).	Signalement envoyé (deux signalements au total).	☐
32	Bob	Saisir `/admin` dans la barre d'adresse.	Redirection vers `/dashboard` (accès refusé aux étudiants).	☐
33	Admin	Se déconnecter (navigateur de Carol) puis se connecter avec le compte administrateur.	Arrivée sur « Publications signalées » : la demande SQL avec ses 2 signalements « En attente ».	☐
34	Admin	Saisir `/dashboard` puis `/requests/new`.	Redirection vers `/admin` (pages réservées aux étudiants).	☐
35	Admin	« Masquer la publication » et confirmer.	« Publication masquée » ; la liste « À traiter » devient vide (« Aucun signalement à traiter »).	☐
36	Bob	Recharger la demande SQL ; ouvrir « Demandes ».	« Demande introuvable » ; elle n'apparaît plus dans la liste.	☐
37	Alice	Ouvrir le tableau de bord.	La demande SQL porte la mention « Masquée par la modération » ; « 0 ouverte(s) ».	☐
38	Admin	« Historique complet » → « Rétablir la publication ».	Signalements « Traité » ; publication de nouveau visible pour les étudiants.	☐
39	Bob	Signaler la demande AVL (« Test de rejet »). Puis Admin : « Rejeter les signalements ».	Signalement « Rejeté » ; la demande AVL reste visible.	☐
G. Interface, persistance, déconnexion				
40	Tous	Bouton lune / soleil dans l'en-tête, puis recharger.	Passage en mode sombre, choix conservé au rechargement.	☐
41	Tous	Réduire la fenêtre (ou outils de développement, 390 px de large).	Menu repliable « ☰ », aucun défilement horizontal, contenu lisible.	☐
42	Tous	Naviguer uniquement au clavier (Tab, Entrée, Espace).	Focus visible ; lien « Aller au contenu » ; formulaires et étoiles utilisables.	☐
43	—	Exécuter `stop.bat` puis `start.bat`, se reconnecter.	Toutes les données (demandes, propositions, évaluations, signalements) sont conservées.	☐
44	Alice	« Se déconnecter », puis ouvrir `/dashboard`.	Redirection vers la page de connexion.	☐

Vérifier que les règles sont bloquées côté serveur

Les règles ne reposent pas sur l'interface. Exemple, dans PowerShell, avec le jeton de Carol (aidante refusée) et l'identifiant de la demande AVL :

```
$login = Invoke-RestMethod -Method Post -Uri http://localhost:8270/api/auth/login `
  -ContentType "application/json" -Body '{"email":"carol@test.tn","password":"MotDePasse123"}'
$h = @{ Authorization = "Bearer $($login.token)" }
Invoke-RestMethod -Method Post -Uri http://localhost:8270/api/requests/1/ratings -Headers $h `
  -ContentType "application/json" -Body '{"score":1}'
# → 403 « Seuls les participants de la séance peuvent l'évaluer »
```

Les 92 tests automatisés (section 3.8) vérifient de la même façon toutes les règles et tous les codes d'erreur.

7 Limites connues et pistes d'amélioration

Limites connues

- **Annulation** : une demande ne peut pas être annulée ni supprimée par son auteur ; une proposition ne peut pas être retirée par l'aidant.
- **Séance non honorée** : seul l'auteur clôture la séance ; il n'existe pas de statut « annulée » si l'aidant ne se présente pas.
- **Évaluations** : une évaluation ne peut pas être modifiée après envoi.
- **Notifications** : aucune notification (email ou temps réel) ; il faut recharger la page pour voir une nouvelle proposition.
- **Jeton** : pas de renouvellement automatique ; après 120 minutes, l'utilisateur doit se reconnecter (il est ramené sur sa page).
- **Recherche** : pas de recherche plein texte (filtres seulement).
- **Erreurs internes** : une erreur imprévue du backend (500) est redirigée vers la page `/error` de Spring, protégée par la sécurité, et apparaît donc comme « Authentification requise ». Autoriser `/error` dans `SecurityConfig` corrigerait l'affichage.
- **Schéma** : géré par `ddl-auto=update` ; une mise en production demanderait des migrations versionnées (Flyway ou Liquibase).
- **Déploiement** : l'application est prévue pour un poste local ; le déploiement (jalon 5) reste à faire.

Pistes (fonctionnalité originale)

- **Recommandation selon les compétences** : mettre en avant, sur le tableau de bord, les demandes ouvertes des matières déclarées dans le profil.
- **Reconnaissance des contributions** : badges (« 5 séances données », « Note ≥ 4,5 ») calculés à partir des évaluations.
- **Séances collectives** : accepter plusieurs aidants ou plusieurs demandeurs sur une même séance.
- **Calendrier** : proposer un créneau précis lors de l'acceptation et l'exporter (.ics).

8 Déclaration d'utilisation d'outils d'IA

Conformément à l'énoncé (« l'utilisation de ressources externes ou d'outils d'IA doit être déclarée »), l'équipe déclare avoir utilisé un assistant d'IA (Claude, d'Anthropic) pour les éléments suivants :

Élément	Contribution de l'IA
Backend — suivi, évaluation, tableau de bord, signalements et administration	Rédaction du code (entités `Rating` / `Report`, services, contrôleurs, DTO, requêtes) et des 4 classes de tests correspondantes, à partir du code existant d'Arbi.
Frontend Next.js	Rédaction des pages, composants, relais d'API, garde des pages, design, logo et mode sombre.
Outillage et documentation	Scripts `start.bat` / `stop.bat`, configurations IntelliJ, documentation de l'API et présent document.

Le socle du backend (authentification, profil, demandes, propositions, acceptation / refus, sécurité et tests associés) a été réalisé par Arbi Hazbri. Les membres de l'équipe ont relu, exécuté et testé le code produit ; chacun doit être capable d'expliquer son fonctionnement et d'y apporter une modification pendant la soutenance.

> À compléter par l'équipe avant la remise : préciser la répartition réelle du travail et les parties que chaque membre présentera.